



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

08 · Flujo de datos

Documentación de sistema · Empresa Operativa Chile

Documento 08 de la serie

Versión del manifiesto 1.5.0 · commit 25f1a5e

Análisis del 2026-09-22

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

08 · Flujo de datos

La conclusión, primero

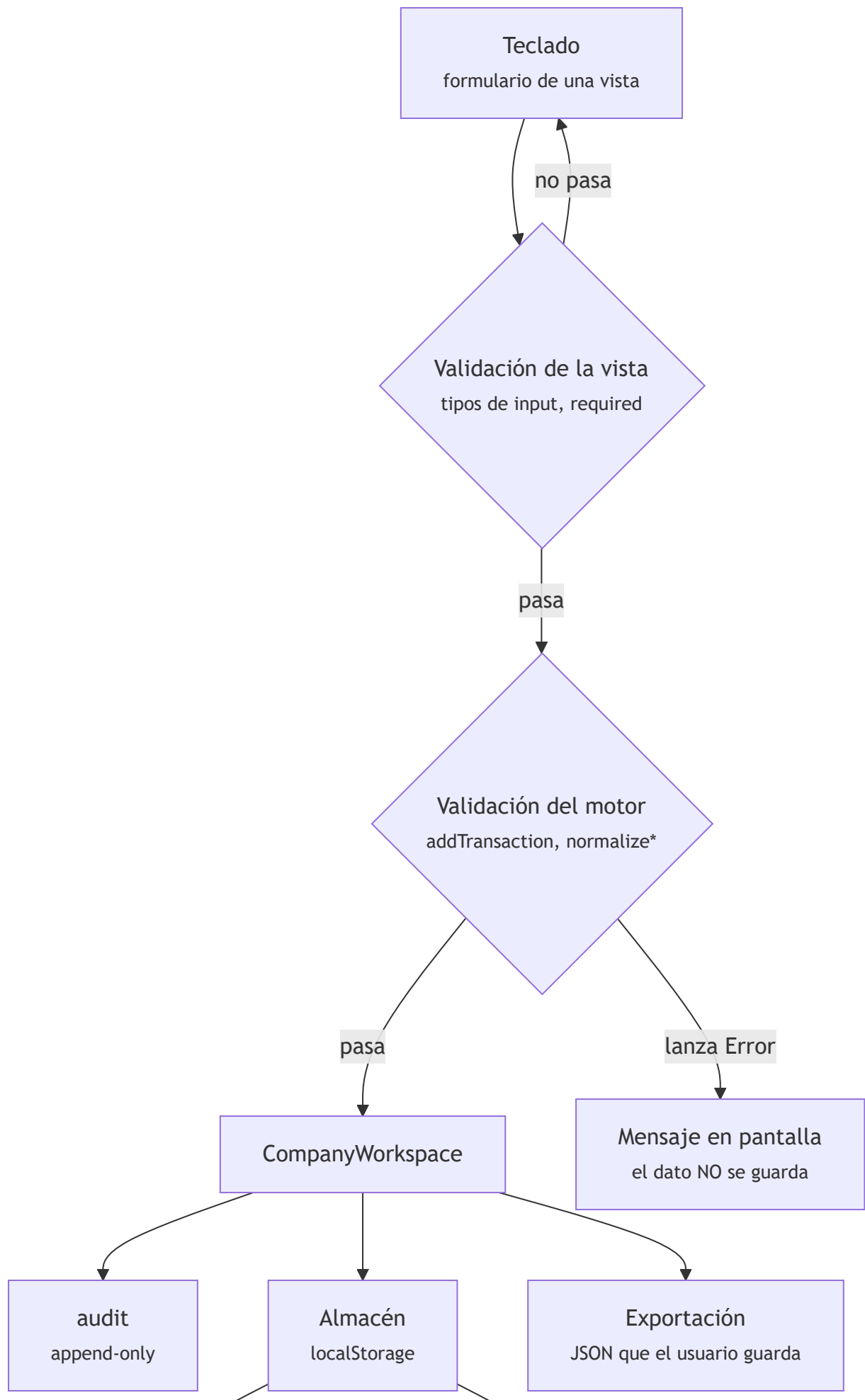
Hay **dos flujos de datos en este sistema y no se tocan**:

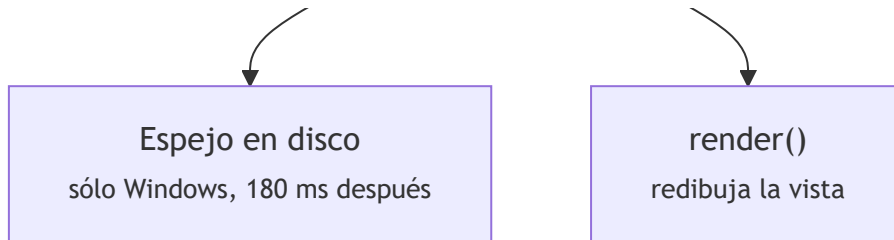
1. **Los datos del usuario** entran por el teclado, se validan, se guardan en el dispositivo y sólo salen si la persona exporta un archivo a propósito. Nunca cruzan la red.

1. **Las reglas tributarias** entran por un *pull request*, se verifican contra una fuente oficial, se compilan al build y viajan **de sólo lectura** dentro del artefacto. Nunca las escribe la app.

El primero es privado y mutable; el segundo es público e inmutable. Que estén separados es lo que permite decir a la vez «tus datos son tuyos» y «las tasas son auditables».

Flujo 1 · Los datos del usuario





Qué muestra: que hay dos capas de validación y que la segunda es la que manda. **Qué no muestra:** que `render()` vuelve a leer del almacén — la vista nunca pinta desde una variable en memoria, siempre relee.

1. De dónde vienen los datos

Origen	Qué aporta	Dónde entra
Formularios de las 14 vistas	Todo lo que el usuario escribe	<code>apps/web/src/views/*.js</code>
Semilla del sandbox	1 empresa, 5 movimientos patrimoniales, 18 operaciones, 4 obligaciones	<code>seedSandboxWorkspace()</code>
Respaldo importado	Un espacio de trabajo completo	<code>importAll()</code>
Escenario JSON de la CLI	Un caso de simulación	<code>data/scenarios/</code>
Espejo del disco (Windows)	Lo que falte en <code>localStorage</code> al arrancar	<code>hydrateFromDisk()</code>
Reglas del año	Tasas, plazos, referencias legales	<code>loadRules(year)</code>
Reloj del sistema	<code>createdAt</code> , <code>updatedAt</code> , <code>at</code> de la bitácora, período por defecto	<code>isoNow()</code>

No hay ninguna otra fuente. No hay importación de RCV, ni de cartola bancaria, ni de DTE, ni lectura de la propuesta del SII. Se declara aquí porque es la pregunta que hace todo el mundo: **NO IDENTIFICADO** — no existe en el repositorio.

2. Cómo se validan

La regla es que **la vista ayuda y el motor decide**. Un formulario puede marcar un campo como requerido, pero la garantía real está en el motor, porque la CLI y las pruebas entran por ahí sin pasar por ninguna vista.

Validación	Dónde	Qué rechaza
Tipo de operación	<code>addTransaction</code>	Un <code>kind</code> fuera de los nueve
Formato de fecha	<code>addTransaction</code> , <code>normalizeEquityMovement</code>	Cualquier cosa que no sea <code>YYYY-MM-DD</code>
Período cerrado	<code>addTransaction</code> , <code>updateTransaction</code> , <code>deleteTransaction</code>	Alta, edición o baja en un mes cerrado
Monto	<code>normalizeEquityMovement</code>	Montos nulos, negativos o no numéricos
Bien aportado	<code>normalizeEquityMovement</code>	Un aporte en bienes sin descripción ni aportante
Coherencia del capital	<code>validateCapitalProfile</code>	Combinaciones imposibles entre social, suscrito y enterado
Evidencia	<code>updateFormationStep</code> , <code>upsertObligation</code>	Dar por hecho algo sin comprobante
Motivo	<code>reopenPeriod</code> , <code>reopenFiscalYear</code>	Reabrir sin explicar por qué
Formato de respaldo	<code>importAll</code>	Un JSON que no es de esta aplicación
RUT	<code>packages/company-operations/rut.mjs</code>	Dígito verificador incorrecto
Modo	Constructor, y otra vez en Rust	Cualquier cosa que no sea <code>real</code> o <code>sandbox</code>

Todas fallan lanzando `Error` , no devolviendo `false` . La consecuencia es que un dato inválido **no llega nunca al almacén**: no hay estado intermedio a medio guardar.

3. Cómo se transforman

Transformación	Función	Qué hace y por qué
Redondeo monetario	<code>money()</code>	Todo importe se guarda como entero de pesos. Sin decimales no hay derivas de centavos al sumar meses
Normalización del capital	<code>normalizeCapitalPosition()</code>	Migra el campo antiguo y marca lo que no se sabe como <code>pendingConfirmation</code>
Normalización del movimiento	<code>normalizeEquityMovement()</code>	Recorta cadenas, fija <code>status</code> y <code>origin</code> por defecto
Derivación del período	<code>date.slice(0, 7)</code>	El período no se guarda : se deriva de la fecha. Una fuente, no dos
Agregación mensual	<code>periodSummary()</code>	18 cifras derivadas de las operaciones del mes
Arrastre del IVA	<code>vatCarryForwardInto()</code>	Recorre la historia desde el principio acumulando remanente
Unión sin duplicar	<code>effectiveEquityMovements()</code>	Fusiona ledger y operaciones de caja usando <code>equityMovementId</code>
Congelado del cierre	<code>closeFiscalYear()</code>	<code>Object.freeze</code> sobre el resultado, con la huella de las reglas dentro

Nada de esto se guarda calculado. `periodSummary`, `capitalPosition`, `estimatedBalance`, `yearSummary` y `healthCheck` se recalculan en cada lectura desde `transactions` y `equity-movements`. La única excepción es deliberada: **lo que queda dentro de un cierre**, mensual o anual, se congela para siempre.

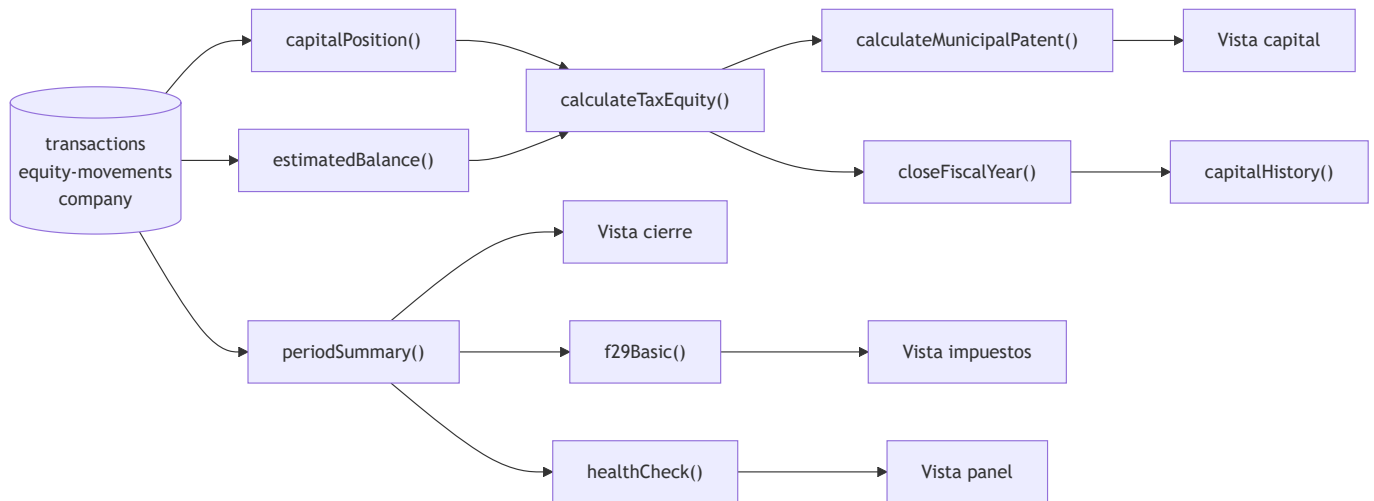
Es el criterio que separa una cifra viva de una cifra histórica. Una vista muestra lo que las operaciones dicen hoy; un cierre muestra lo que decían el día que se cerró. Si todo se recalculara, un cierre no probaría nada.

4. Dónde se almacenan

El detalle completo está en [07 · Persistencia](#). En una línea: `localStorage` con prefijo por modo, más un espejo en JSON bajo Windows, más archivos por entidad cuando el consumidor es Node.

El espejo de Windows llega tarde a propósito. `createPlatformStore` no escribe a disco en cada `write`: agenda un volcado a los **180 ms** y cancela el anterior. Diez escrituras seguidas producen un volcado, no diez. Y si el disco falla, **la aplicación sigue funcionando** con `localStorage` y sólo avisa por consola: perder la sesión por un error de escritura sería peor que perder el espejo.

5. Qué componentes los consumen



Qué muestra: que casi todo el sistema depende de dos colecciones, y que el CPT es el cuello por el que pasa la cadena capital → cierre → patente. **Qué no muestra:** las 14 vistas completas; sólo las que consumen datos derivados.

6. Qué se muestra al usuario

Lo que la interfaz pinta **nunca es el dato crudo**: es el resultado de las derivaciones anteriores. Y hay tres decisiones de presentación que son reglas de negocio disfrazadas:

- **balanceOrigin** viaja hasta la pantalla: la aplicación dice si el balance lo estimó ella o lo declaró el usuario. No presenta una estimación como un dato.
- **pendingConfirmation** se muestra: la app señala qué campos **sabe que no sabe**, en vez de rellenarlos con cero.
- **healthCheck** deliberadamente **no dice «todo en orden»** cuando faltan evidencias. El objetivo es detectar el hueco, no tranquilizar.

7. Qué sale del dispositivo

Nada, salvo que el usuario lo pida. Tres salidas, todas iniciadas por una acción explícita:

Salida	Mecanismo	Destino
Respaldo / exportación	<code>saveTextFile()</code>	En Windows, Documentos/Empresa Operativa Chile ; en web y Android, la descarga del navegador
Enlace a un portal oficial	<code>openExternal()</code>	El navegador del sistema, o el plugin Browser de Capacitor en Android
Nada más	—	—

openExternal merece una nota: abre el sitio del SII o del Registro de Empresas **fuera** de la aplicación, y con **noopener, noreferrer** . La app no enmarca portales oficiales ni les pasa datos: sólo lleva a la persona hasta la puerta.

No hay telemetría, ni analítica, ni informe de errores, ni comprobación de actualizaciones, ni fuentes o iconos remotos. La comprobación está en 09 y en 11.

8. Dónde puede perderse o corromperse un dato

Esta sección existe porque el prompt la pide y porque las respuestas honestas son incómodas.

Punto	Qué puede pasar	Mitigación presente	Mitigación ausente
«Borrar datos del sitio» en el navegador	Pérdida total	Ninguna	La app no puede impedirlo
Desinstalar el APK	Pérdida total	Ninguna	Idem
Cuota de <code>localStorage</code> agotada	<code>setItem</code> lanza y la escritura se pierde	Ninguna: no se captura <code>QuotaExceededError</code>	Ver 15 · Riesgos
Corte durante el espejo de Windows	Espejo a medias	Escritura atómica: temporal + <code>rename</code> en Rust	—
JSON corrupto en <code>localStorage</code>	<code>read</code> devuelve el <code>fallback</code>	El <code>try/catch</code> evita el fallo	Silencioso: no distingue vacío de corrupto
Línea corrupta en <code>audit.ndjson</code>	Se devuelve <code>{ corrupt: true, line }</code>	La fila mala no rompe la lectura	Nadie muestra ese marcador
Cierre a medias	Fotografía sin índice	Ninguna: son dos escrituras	No hay transacción
Dos pestañas del navegador a la vez	La última escritura gana	Ninguna	No hay bloqueo ni fusión
Reloj del sistema mal puesto	Fechas y bitácora incoherentes	Ninguna	No hay fuente de tiempo alternativa

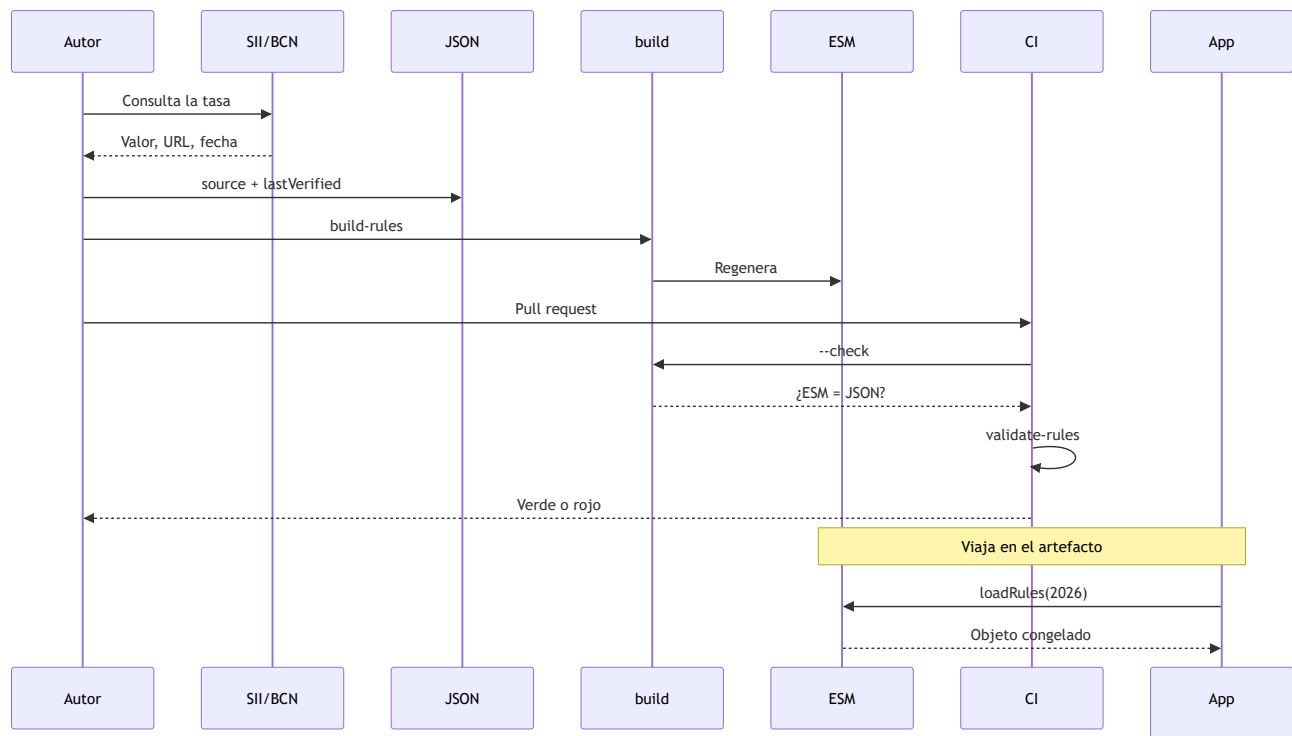
De todos, el único que la aplicación puede resolver por sí sola es el respaldo: `SECURITY.md` insiste en exportar al cerrar cada período, y ese consejo es la contrapartida real de no tener servidor.

9. Qué datos personales se procesan

RUT de la empresa y de terceros, nombres y RUT de accionistas, montos, folios, descripciones de operaciones y comuna. Todos **en claro**, todos **sólo en el dispositivo**, ninguno enviado a nadie. El detalle y sus implicaciones están en 07 y 11.

Flujo 2 · El ciclo de vida de una regla tributaria

El otro flujo, el que no toca al usuario.



Las tres puertas que protegen este flujo, y qué atrapa cada una:

Puerta	Comando	Qué impide
Sincronía	<code>build-rules.mjs --check</code>	Que alguien edite el <code>.json</code> y olvide regenerar, o que edite el generado a mano
Cordura	<code>validate-rules.mjs</code>	Un error de dedo: un 19 donde iba 0.19, una tasa fuera de rango
Existencia	<code>loadRules(year)</code>	Calcular con las tasas del año equivocado

Y la garantía que no está en el flujo, sino en el dato: cada tasa lleva `source` y `lastVerified`. La aplicación no sólo sabe cuánto es el IVA — sabe **de dónde lo sacó y cuándo lo comprobó alguien**. Un cierre anual se lleva esa huella dentro (`legalRulesVersion`), de modo que la trazabilidad sobrevive a que el archivo de reglas cambie después.

Lo que este flujo no tiene: ninguna comprobación automática contra la fuente. Nadie despierta un lunes a preguntarle al SII si la tasa cambió; `lastVerified` es una afirmación humana. **REQUIERE VALIDACIÓN** permanente: las tasas de 2026 hay que reverificarlas antes de la Operación Renta 2027, y el propio archivo lo dice.

Flujo 3 · Los datos que generan la documentación

Un tercer flujo, menor pero real: parte de la documentación del repositorio **no se escribe, se deriva del código**.

Fuente	Script	Salida	Gate en CI
packages/glossary/	build-glossary.mjs	docs/GLOSSARY.md	--check
packages/onboarding/	build-guide.mjs	docs/EMPEZAR-AQUI.md/.html/.pdf	--check
packages/shortcuts/	build-shortcuts.mjs	docs/ATAJOS-DE-TECLADO.md	--check
docs/presentacion.md	build-presentation.mjs	Diapositivas HTML y PDF	--check + check-presentation.mjs
docs/MANUAL.md + capturas	build-manual.mjs	docs/MANUAL.html/.pdf	—

Por qué importa para el flujo de datos: si alguien edita docs/GLOSSARY.md a mano, CI falla. El documento no es la fuente; el paquete lo es. Es el mismo principio que las reglas tributarias aplicado a las palabras: un texto editado en dos sitios acaba diciendo dos cosas.

Un ejemplo completo, de principio a fin

Registrar una venta de \$100.000 netos el 18 de julio de 2026, en Windows:

#	Qué ocurre	Dónde	Dato
1	La vista propone el IVA al escribir el neto	<code>views/operacion.es.js</code>	<code>19000</code>
2	Se envía el formulario	<code>views/operacion.es.js</code>	<code>{ kind: 'sale', date: '2026-07-18', net: 100000, vat: 19000 }</code>
3	Tipo válido, fecha con formato, período <code>2026-07</code> no cerrado	<code>addTransaction</code>	pasa
4	Se completan los valores por defecto	<code>addTransaction</code>	<code>deductible: true , vatCreditEligible: true , total: 119000 , id , createdAt</code>
5	Se escribe la colección completa	<code>createWebStore.write</code>	<code>empresa-operativa-chile:real:transactions</code>
6	Se agrega la línea de bitácora	<code>audit()</code>	<code>transaction.added</code> con id, kind, total y período
7	Se agenda el espejo	<code>createPlatformStore</code>	a los 180 ms
8	Rust valida el JSON y escribe atómicamente	<code>save_workspace</code>	<code>real.json.tmp</code> → <code>real.json</code>
9	La vista se redibuja leyendo del almacén	<code>render()</code>	la operación aparece en la lista
10	El mes cambia sin que nadie lo recalculé	<code>periodSummary('2026-07')</code>	<code>salesNet: 100000 , debitVat: 19000</code>
11	El borrador del F29 cambia con él	<code>f29Basic()</code>	débito fiscal <code>19000</code> , menos el crédito del mes y el remanente que llegue

Once pasos, cero llamadas de red y dos archivos escritos. Ese es el sistema entero.